pass the query a `datetime` object. All command-line arguments are strings, so before you create your query, you need to convert the command-line flag into a date. Dates are problematic, as their format differs between regions. As a programmer, my default for handling this problem is to make everyone uncomfortable and require the year-month-day format (it's the only sane value numerically, and it's a standard: ISO 8601).

It is all well and good to write "just use my format," but of course, users don't always pay attention. There is a chance your user won't read the instructions and may pass you a date that `datetime` can't parse. You could just let `datetime` throw an error and get the resulting ugly stack trace, or you could catch the `datetime` error and throw your own. Django's management commands have their own exception: `django.core.management.base.CommandError`. If you throw one of these, Django formats a pretty error message and even color codes it if the user's terminal supports it.

Listing 9.3 has the updated code for the `musicians` management command, which now supports filtering based on the musician's first name, last name, and date of birth. For each of the three filters, a command line flag is available, so the user can specify the filter. To register a flag, you call `.add_argument()` on the parse object in the command's `.add_arguments()` method. Inside the `.handle()` method, you query the musicians and modify the query based on which flags got included. The `.filter()` query method supports field-lookup modifiers, allowing you to filter the contents based on the `Musician Model` fields. The `options` object contains the values of any flags used when a management command gets invoked.

Listing 9.3 Add filtering to your `musicians` management command

```
# RiffMates/bands/management/commands/musicians.py
from datetime import datetime
from django.core.management.base import BaseCommand
from django.core.management.base import CommandError
from bands.models import Musician

class Command(BaseCommand):
    help = "Lists registered musicians"

    def add_arguments(self, parser):
        parser.add_argument("--last_name", "-l",
            help=(
                "Query musicians whose last name is greater than or equal"
                " to this value. Note this is case sensitive."
            )
        )
        parser.add_argument("--first_name", "-f", help=("Query musicians "
            "whose first name is greater than or equal to this value. "
            "Note this is case sensitive."))
        parser.add_argument("--birth", "-b", help=("Query musicians "
            "whose birth date is greater than or equal to this value. "
            "Date must be given in YYYY-MM-DD format."))

    def handle(self, *args, **options):
```

Implement .add_arguments() to register command-line positional arguments and flags.

The parser object passed into .add_arguments() is an argparse parser. Use its .add_argument() method to register a flag. Flags are indicated by leading dashes.

The help parameter gets used by .add_argument() to add flag-specific information to the help screen.